



ADDIS ABABA SCIENCE AND TECHNOLOGY UNIVERSITY

College of Engineering

Department of Electrical and Computer Engineering

New Trend Course — Individual Assignment

The Codeless Revolution:
AI-Powered No-Code / Low-Code Platforms
Enabling Custom Application Development Beyond
Traditional Programming Boundaries in 2026

Name

Netsanet Abuhay

ID No

ETS 1301/14

Submitted to: _____

submission date: May 29 , 2026

TABLE OF CONTENTS

Abstract	4
Chapter 1: Introduction	5
1.1 Background	5
1.2 Problem Statement.....	5
1.3 Objectives	5
1.4 Scope of the Study	6
1.5 Significance of the Study	6
Chapter 2: Literature Review.....	7
2.1 Historical Evolution of No-Code / Low-Code Development	7
2.2 The AI Integration Era (2023–2026)	7
2.3 Key Academic and Industry Research.....	7
2.4 Related Open-Source Projects	8
Chapter 3: AI-Powered No-Code / Low-Code Platforms.....	9
3.1 Definition and Core Concepts.....	9
3.2 Architecture of Modern No-Code Platforms	9
3.3 How AI Enhances No-Code Platforms.....	10
3.3.1 Natural Language Processing.....	10
3.3.2 Intelligent Component Suggestions	10
3.3.3 Automatic Error Detection.....	10
3.3.4 Adaptive Learning.....	10
3.4 Market Analysis and Industry Impact in 2026	10
3.5 Leading Platforms in the 2026 Ecosystem.....	11
Chapter 4: Flowise — Open Source Case Study	12
4.1 Overview of Flowise	12
4.2 Key Features.....	12
4.3 Technical Architecture.....	13
4.3.1 Server Package	13
4.3.2 UI Package.....	13
4.3.3 Components Package	13
4.3.4 Technology Stack	13
4.4 Codebase Statistics.....	13
4.5 License and Open Source Model	14
Chapter 5: Installation and Testing.....	15
5.1 Development Environment	15
5.2 Forking the Repository	15
5.3 Cloning the Repository	15
5.4 Version Compatibility Resolution	16
5.5 Dependency Installation	16
5.6 Building the Project	16
5.7 Starting the Application.....	16
Chapter 6: Results and Discussion	17
6.1 Successful Installation Outcomes.....	17

6.2 Flowise User Interface Observations	17
6.2.1 Main Dashboard	17
6.2.2 Visual Canvas.....	17
6.2.3 Component Sidebar	17
6.3 Demonstration: Simple RAG Chatbot Construction	17
6.4 Performance Observations on 4GB RAM System	18
6.5 Evaluation Against Study Objectives	18
Chapter 7: Conclusion and Future Work.....	19
7.1 Conclusion	19
7.2 Limitations	19
7.3 Future Work	19
7.4 Final Reflection	20
References	21

Abstract

This individual assignment report presents a comprehensive academic exploration of AI-Powered No-Code and Low-Code platforms as one of the most transformative emerging technologies of 2026. The study is centered on the open-source platform Flowise, a visually-driven AI agent builder that enables users to construct sophisticated AI workflows, chatbots, and autonomous agents without requiring traditional programming skills. The report investigates the theoretical foundations, architectural design, and real-world applications of no-code and low-code technologies, demonstrating how these platforms are eliminating traditional barriers to software development. Through a detailed installation, configuration, and testing process conducted on an Ubuntu Linux environment using Node.js v20.20.2 and npm v10.26.0, the study provides first-hand technical evidence of how the Flowise platform operates. The findings reveal that AI-powered no-code platforms represent a fundamental paradigm shift in software engineering, democratizing application development and making it accessible to individuals without programming expertise. The report concludes with reflections on future directions and the long-term impact of codeless development technologies on society, industry, and education.

Chapter 1: Introduction

1.1 Background

Software development has historically been a domain restricted to individuals with formal training in computer science or programming. The ability to create functional applications required mastery of complex programming languages such as Python, Java, JavaScript, or C++, which demanded years of study and practice. This traditional barrier has consistently limited the pace of digital transformation, particularly in developing nations and resource-constrained environments where access to skilled developers is limited.

In recent years, a technological revolution has been quietly reshaping this landscape. The emergence of No-Code and Low-Code development platforms has fundamentally challenged the assumption that software creation requires programming expertise. These platforms offer intuitive visual interfaces, drag-and-drop components, and pre-built logic blocks that allow non-technical users to build fully functional applications. The integration of Artificial Intelligence (AI) into these platforms in 2025 and 2026 has further accelerated this transformation, enabling platforms to understand natural language instructions, generate code automatically, and suggest intelligent workflow designs.

Flowise, the open-source platform selected for this study, represents a state-of-the-art example of AI-powered no-code development. Available at <https://flowiseai.com> and fully open-source on GitHub at <https://github.com/FlowiseAI/Flowise>, Flowise enables users to construct AI agents and workflows through a visual drag-and-drop canvas, connecting AI models with data sources, tools, and external services — all without writing a single line of code.

1.2 Problem Statement

Despite rapid advancements in artificial intelligence and software technology, a significant portion of the global population remains unable to participate in the digital economy as creators rather than consumers. The complexity and cost of traditional software development creates a digital divide where only organizations with large technical teams and substantial budgets can build custom software solutions. This exclusion limits innovation, slows economic development, and prevents individuals from solving problems in their own communities through technology.

Furthermore, the demand for software continues to grow at a pace that far exceeds the supply of trained developers. According to industry estimates, there will be a shortage of over 85 million technology workers globally by 2030. No-code and low-code platforms powered by AI represent a critical solution to this challenge by enabling a broader population to participate in software creation.

1.3 Objectives

This report aims to achieve the following objectives:

1. To provide a thorough academic review of AI-powered no-code and low-code platform technology as it stands in 2026.
 2. To analyze Flowise as a representative open-source case study of this technology category.
 3. To document the complete installation, configuration, and testing process of Flowise on a Linux Ubuntu environment.
 4. To evaluate the capabilities, limitations, and future potential of codeless application development platforms.
-

-
5. To demonstrate practical understanding of the technology through hands-on experimentation and documentation.

1.4 Scope of the Study

This report focuses specifically on AI-powered no-code and low-code development platforms within the context of 2026 technological trends. The study examines the Flowise open-source platform as a primary case study, including its architecture, features, installation process, and operational capabilities. The scope includes both theoretical analysis and practical experimentation conducted on an Ubuntu Linux system. The report does not cover commercial proprietary no-code platforms in technical depth, nor does it address mobile-specific or gaming-specific development environments.

1.5 Significance of the Study

This study is significant for several reasons. First, it contributes to the academic understanding of a rapidly evolving technology that is reshaping the software industry. Second, it provides practical documentation that can serve as a guide for other students and practitioners seeking to explore no-code AI platforms. Third, by examining an open-source implementation, the study promotes awareness of freely available tools that can benefit educational institutions, startups, and organizations in developing economies such as Ethiopia, where access to expensive commercial software is limited.

Chapter 2: Literature Review

2.1 Historical Evolution of No-Code / Low-Code Development

The concept of visual programming and abstracted development tools is not new. The roots of no-code development can be traced back to the early 1980s when tools such as HyperCard (Apple, 1987) allowed users to create interactive applications through a card-based interface without programming. Similarly, Visual Basic (Microsoft, 1991) introduced event-driven programming with a visual designer, enabling a generation of developers to build Windows applications with less code than traditional approaches required.

The 2000s saw the emergence of web-based content management systems such as WordPress (2003) and Joomla (2005), which allowed non-programmers to build websites through visual editors. These systems represented an early form of no-code development, though they were limited in functionality compared to custom-coded applications.

The formal no-code and low-code movement gained momentum in the 2010s with the launch of platforms such as Salesforce's Force.com, OutSystems, and Mendix. Research firm Forrester coined the term 'low-code' in 2014 to describe platforms that minimized manual coding through visual development environments. By 2017, Gartner had identified low-code application platforms (LCAP) as a distinct and rapidly growing technology category.

2.2 The AI Integration Era (2023–2026)

The most significant development in the no-code and low-code space has been the integration of generative AI capabilities beginning in 2023. The emergence of large language models (LLMs) such as GPT-4, Claude, and Gemini created new possibilities for natural language-driven development. Users could now describe what they wanted in plain English, and the AI would generate the appropriate application structure, logic, or code automatically.

By 2025 and 2026, this AI integration had matured significantly. Platforms began offering features such as AI-generated workflow suggestions, automatic error detection and correction, intelligent data mapping, and conversational interfaces for application building. The result was a new generation of AI-powered no-code platforms that were substantially more powerful and accessible than their predecessors.

Flowise, launched as an open-source project in 2023 and significantly updated through 2025 and 2026, exemplifies this AI-native approach to no-code development. It was designed from the ground up to enable the visual construction of AI workflows, making it particularly relevant to the current technological moment.

2.3 Key Academic and Industry Research

Several important studies have contributed to our understanding of no-code and low-code platforms. Richardson and collaborators at Gartner (2021) established foundational taxonomies for the low-code market, distinguishing between platforms focused on professional developers versus citizen developers. Their research predicted that by 2025, 70% of new applications would be developed using low-code or no-code technologies.

Rokis and Kirikova (2022) conducted a systematic literature review of low-code development challenges, identifying maintainability, scalability, and vendor lock-in as primary concerns. Their findings are particularly relevant for enterprise adoption considerations.

Sahay and colleagues (2020) explored the concept of 'citizen development' enabled by no-code platforms, finding that organizations that adopted citizen development programs

achieved 30-50% faster application delivery and significant cost savings compared to traditional development approaches.

More recently, industry research from Forrester (2026) has highlighted the transformative role of AI in no-code platforms, noting that AI-augmented low-code platforms reduce the cognitive load on citizen developers by automating up to 60% of design decisions through intelligent suggestions.

2.4 Related Open-Source Projects

Several open-source projects operate in the same space as Flowise. Appsmith (<https://www.appsmith.com>) provides an open-source framework for building internal business tools with a visual interface. Budibase offers a self-hosted low-code platform for building applications on top of databases. n8n provides workflow automation similar to Zapier but with full self-hosting capability. NocoDB transforms any database into a smart spreadsheet interface without code.

What distinguishes Flowise from these alternatives is its specific focus on AI agent and workflow construction. While other platforms build general-purpose business applications, Flowise is purpose-built for constructing AI pipelines, making it uniquely positioned at the intersection of no-code development and artificial intelligence — the defining technological intersection of 2026.

Chapter 3: AI-Powered No-Code / Low-Code Platforms

3.1 Definition and Core Concepts

AI-Powered No-Code and Low-Code platforms are software development environments that enable users to create fully functional applications, automated workflows, and AI-driven systems through visual interfaces rather than traditional text-based programming. These platforms abstract the complexity of software engineering into intuitive graphical components that can be assembled, connected, and configured without writing code.

The term 'No-Code' refers to platforms where absolutely no programming knowledge is required — users interact entirely through graphical interfaces. 'Low-Code' platforms, by contrast, may require minimal coding for advanced customization but still dramatically reduce the total amount of code that must be written compared to traditional development approaches. The addition of AI capabilities in 2025-2026 has further enhanced both categories by enabling natural language interfaces and intelligent automation.

3.2 Architecture of Modern No-Code Platforms

Modern AI-powered no-code platforms typically consist of several architectural layers working in concert:

- **Visual Builder Layer:** The front-end interface where users drag, drop, and connect components on a canvas. This layer translates visual configurations into underlying application logic without exposing code to the user.
- **Component Library:** A curated collection of pre-built functional blocks representing common application elements such as data inputs, logic processors, API connectors, AI model integrations, and output handlers.
- **Execution Engine:** The backend runtime that interprets the visual configuration and executes the corresponding logic. In AI-powered platforms, this engine may include LLM orchestration frameworks such as LangChain or LlamaIndex.
- **Integration Layer:** APIs and connectors that link the platform to external services, databases, and AI models. Modern platforms offer hundreds of pre-built integrations.
- **AI Intelligence Layer:** The generative AI subsystem that provides natural language understanding, automatic code generation, intelligent suggestions, and error correction capabilities.

The following table summarizes the key characteristics of different no-code platform architectures:

Architecture Type	Target Users	AI Integration	Example Platforms
Visual Flow Builder	Citizen Developers	LLM Orchestration	Flowise, n8n
App Builder	Business Analysts	Auto-generation	Bubble, Glide
Database-Centric	Data Teams	Smart Analytics	NocoDB, Airtable
Process Automation	Operations Teams	Workflow AI	Zapier, Power Automate
Enterprise LCAP	Professional Devs	Full AI Suite	OutSystems, Mendix

3.3 How AI Enhances No-Code Platforms

The integration of artificial intelligence into no-code platforms has transformed them from simple visual builders into intelligent development assistants. Key AI enhancements include:

3.3.1 Natural Language Processing

Users can describe what they want in plain language and the AI interprets the description to suggest or automatically construct the appropriate workflow. For example, a user might type 'Create a customer support chatbot that reads from our FAQ document and escalates unanswered questions to a human agent,' and the AI will generate the corresponding flow automatically.

3.3.2 Intelligent Component Suggestions

As users build workflows, AI analyzes the context and suggests the most appropriate next components. This reduces decision fatigue and guides inexperienced users toward best practices.

3.3.3 Automatic Error Detection

AI models continuously monitor workflows for logical errors, missing configurations, and potential failure points, providing real-time feedback before deployment. This quality assurance function traditionally required experienced developers.

3.3.4 Adaptive Learning

Enterprise platforms are beginning to incorporate user behavior analysis, where the AI learns from how individual teams build applications and personalizes its suggestions to match organizational patterns and preferences.

3.4 Market Analysis and Industry Impact in 2026

The economic significance of no-code and low-code platforms has grown dramatically. Key market indicators for 2026 include:

Metric	Value	Source
Global Market Size (2026)	\$44.5 Billion	Forrester Research
Annual Growth Rate	19% CAGR	Gartner
% New Apps Using LCNC	75%	Gartner
Development Time Reduction	45%	IDC
Cost Reduction vs Traditional	20–60%	McKinsey
Developer Shortage Addressed	~4M roles	World Economic Forum

These figures demonstrate that no-code and low-code platforms have moved far beyond experimental adoption into mainstream enterprise deployment. The \$44.5 billion market size represents a fivefold increase from 2021, reflecting the accelerating pace of adoption across all industry sectors.

3.5 Leading Platforms in the 2026 Ecosystem

The no-code and low-code ecosystem in 2026 is populated by a diverse range of platforms serving different user needs:

Platform	Category	Primary Use Case	Official Website
Flowise	Open Source / AI	AI Agent & Workflow Builder	https://flowiseai.com
Microsoft Power Apps	Enterprise / Low-Code	Business App Development	https://powerapps.microsoft.com
Bubble	No-Code / SaaS	Web App Development	https://bubble.io
OutSystems	Enterprise / Low-Code	Enterprise Application Dev	https://www.outsystems.com
Zapier	No-Code / Automation	Workflow Automation	https://zapier.com
Budibase	Open Source	Internal Tool Builder	https://budibase.com

Chapter 4: Flowise — Open Source Case Study

4.1 Overview of Flowise

Flowise is an open-source, low-code tool designed specifically for building customized AI agents and workflows using a visual drag-and-drop interface. Created by the FlowiseAI team and maintained as a community-driven open-source project, Flowise enables developers, data scientists, and non-technical users alike to construct sophisticated AI pipelines without deep programming expertise.

The platform is built on top of LangChain, an open-source framework for developing applications powered by language models, and extends it with a visual interface that eliminates the need to write LangChain code manually. Users can chain together AI models, memory modules, document loaders, vector stores, and external tools through a graphical interface, significantly reducing the time required to prototype and deploy AI applications.

Official Website: <https://flowiseai.com>

Documentation: <https://docs.flowiseai.com>

GitHub Repository (Original): <https://github.com/FlowiseAI/Flowise>

My Forked Repository: <https://github.com/netsanetabuhay/Flowise>

Cloud Platform: <https://cloud.flowiseai.com>

Community Discord: <https://discord.gg/jn8n9yGZFr>

4.2 Key Features

Flowise offers an extensive set of features that make it a powerful platform for AI application development:

- **Visual Drag-and-Drop Canvas:** An intuitive workspace where users connect AI components through a node-based interface. Each node represents a specific functionality such as an LLM, a document loader, a vector store, or a tool.
 - **100+ Pre-Built Integrations:** Native support for major AI providers including OpenAI, Anthropic Claude, Google Gemini, HuggingFace, Ollama (for local models), and many others.
 - **Document Intelligence (RAG):** Built-in support for Retrieval-Augmented Generation, allowing users to upload documents in PDF, Word, CSV, and other formats and create AI systems that answer questions based on that content.
 - **Multi-Agent Orchestration:** Advanced capability to build systems where multiple AI agents collaborate, with supervisor agents directing worker agents to complete complex multi-step tasks.
 - **API Generation:** Every workflow built in Flowise automatically generates a REST API endpoint, making it instantly deployable and integrable with external applications.
 - **Embedded Chatbot Widget:** Built-in capability to embed AI chatbots directly into websites with minimal configuration.
 - **Tool Integration:** Support for external tools including web search, calculators, code execution, and custom function definitions.
 - **Credential Management:** Secure storage and management of API keys and authentication credentials.
 - **Scheduling:** Automated execution of workflows on defined schedules.
-

- Self-Hosted: Full control over data and infrastructure with Docker support for easy deployment.

4.3 Technical Architecture

Flowise follows a monorepo architecture organized into distinct packages, each serving a specific role in the overall system:

4.3.1 Server Package

The server package is a Node.js backend application that serves all API logic. It handles request routing, authentication, workflow execution, database operations, and integration with external AI services. The server is written primarily in TypeScript and uses Express.js as the web framework. It supports multiple database backends including SQLite (default for development), PostgreSQL, MySQL, and MariaDB for production deployments.

4.3.2 UI Package

The UI package is a React.js frontend application that provides the visual drag-and-drop canvas. It uses the ReactFlow library for the node-based workflow editor and implements a component library built on top of Material UI. The UI communicates with the server through a RESTful API and supports real-time updates through Server-Sent Events (SSE).

4.3.3 Components Package

The components package contains over 100 integration nodes, each representing a connection to an external AI service, database, tool, or utility. Each component is self-contained with its own configuration schema, credential requirements, and execution logic. This modular design makes it straightforward to add new integrations without modifying the core platform.

4.3.4 Technology Stack

Layer	Technology	Version	Purpose
Backend Runtime	Node.js	v20.x	Server-side JavaScript execution
Backend Framework	Express.js	4.x	REST API routing and middleware
Frontend Framework	React.js	18.x	Visual canvas and UI components
AI Orchestration	LangChain	0.1.x	LLM chaining and agent logic
Build System	Turbo	1.x	Monorepo build orchestration
Package Manager	pnpm	10.26.0	Fast, disk-efficient package management
Language	TypeScript	5.x	Type-safe JavaScript development
Database (Default)	SQLite	5.x	Local development database

4.4 Codebase Statistics

The Flowise codebase is substantial, reflecting the platform's comprehensive feature set. Analysis of the forked repository at <https://github.com/netsanetabuhay/Flowise> reveals the following statistics:

- Total Directories: 697

-
- Total Files: 2,412
 - Primary Language: TypeScript (61.7%)
 - Secondary Language: JavaScript (27.4%)
 - HTML: 5.6%
 - Handlebars Templates: 5.0%
 - CSS/SCSS: 0.3%

The dominance of TypeScript in the codebase reflects a deliberate architectural decision to prioritize type safety and developer tooling. TypeScript's static type checking catches potential errors at compile time rather than runtime, making the codebase more maintainable as it grows in size and complexity.

4.5 License and Open Source Model

Flowise is released under the Apache License Version 2.0, one of the most permissive and widely-used open-source licenses. This license allows users to freely use, modify, distribute, and build upon the software for both commercial and non-commercial purposes. The primary requirement is that the original copyright notice and license text be preserved in any distribution. This licensing model has been instrumental in driving adoption across diverse organizations, from startups to large enterprises.

Chapter 5: Installation and Testing

5.1 Development Environment

The installation and testing process was conducted on the following system configuration:

Component	Specification
Operating System	Ubuntu Linux (ThinkPad Yoga 260)
Hostname	netsanet-ThinkPad-Yoga-260
Node.js Version	v20.20.2 (downgraded from v24 for compatibility)
pnpm Version	v10.26.0 (downgraded from v11 for compatibility)
Code Editor	Visual Studio Code
Shell	Bash (default Ubuntu terminal)
RAM	4 GB

5.2 Forking the Repository

The first step in the installation process was forking the official Flowise repository on GitHub. Forking creates a personal copy of the repository under the student's GitHub account, allowing modifications and documentation to be added while maintaining a connection to the original upstream project.

6. Navigated to the official Flowise repository at <https://github.com/FlowiseAI/Flowise>
7. Clicked the 'Fork' button in the top-right corner of the repository page
8. Selected the student's GitHub account (netsanetabuhay) as the fork destination
9. Waited for GitHub to complete the fork operation
10. Confirmed the fork was created at <https://github.com/netsanetabuhay/Flowise>

The forked repository includes all 2,412 files across 697 directories, representing the complete Flowise codebase at the time of forking.

5.3 Cloning the Repository

After forking, the repository was cloned to the local development machine using Git:

```
$ mkdir flowise-project
$ cd flowise-project
$ git clone https://github.com/netsanetabuhay/Flowise.git
$ cd Flowise
```

The cloning process successfully downloaded all 42,553 objects (59.93 MB) from the remote repository, resolving 26,656 deltas to reconstruct the complete project history.

5.4 Version Compatibility Resolution

Upon attempting the initial installation, version incompatibility errors were encountered. Flowise requires specific versions of both Node.js and pnpm that differ from the latest available versions:

Tool	System Version	Required Version	Action Taken
Node.js	v24.15.0	^20.x	Downgraded to v20.20.2 using 'n' version manager
pnpm	v11.5.0	^10.26.0	Downgraded to v10.26.0 using <code>npm install -g pnpm@10.26.0</code>

The resolution process required installing the Node.js version manager 'n' (`sudo npm install -g n`) and then using it to install and activate Node.js v20.20.2 (`sudo n 20`). The pnpm package manager was similarly downgraded to the exact version specified in the project's package.json engine requirements.

5.5 Dependency Installation

With the correct tool versions in place, the pnpm install command was executed to download and install all project dependencies:

```
|$ pnpm install
```

This command resolved, downloaded, and installed 4,240 packages across all seven workspace projects in the monorepo. Notable large packages downloaded included onnxruntime-node (31.17 MB), turbo-linux-64 (19.81 MB), geoip-lite (43.87 MB), and several ML-specific native binaries. The total installation process completed in 3 minutes and 55 seconds, demonstrating efficient dependency resolution by pnpm's content-addressable storage system.

5.6 Building the Project

Following successful dependency installation, the project was compiled using the build command:

```
|$ pnpm build
```

The build process compiled all TypeScript source files to JavaScript, bundled the React frontend application, and prepared all packages for runtime execution. The Turbo build system orchestrated the build process across all workspace packages in the optimal order, leveraging caching to minimize redundant compilation steps.

5.7 Starting the Application

The application was started using:

```
|$ pnpm start
```

Upon successful startup, the Flowise server begins listening on port 3000. The application is accessible by navigating to `http://localhost:3000` in a web browser. The startup process initializes the database, loads all component nodes from the components package, and starts the Express.js HTTP server.

Chapter 6: Results and Discussion

6.1 Successful Installation Outcomes

The installation process was completed successfully after resolving the version compatibility issues described in Chapter 5. The key outcomes of the installation phase are summarized below:

- All 4,240 npm packages were successfully installed with zero critical errors.
- The project build completed without compilation errors, confirming code integrity.
- The Flowise server started successfully and became accessible at `http://localhost:3000`.
- The visual drag-and-drop canvas loaded correctly in the browser interface.
- The component library displayed all 100+ available AI integration nodes.

6.2 Flowise User Interface Observations

Upon accessing the Flowise interface at `http://localhost:3000`, the following key interface components were observed and evaluated:

6.2.1 Main Dashboard

The main dashboard presents a clean, organized view of all saved chatflows and agent flows. It provides options to create new flows, import existing ones, and access the marketplace of pre-built templates. The interface design is intuitive and requires no technical background to navigate.

6.2.2 Visual Canvas

The visual canvas is the core working area of Flowise. It presents an infinite scrollable workspace where nodes can be placed and connected. Each node represents a specific AI component and can be configured through a settings panel that appears when the node is selected. The connection system uses directional arrows to show data flow between components, making the execution logic visually comprehensible.

6.2.3 Component Sidebar

The left sidebar provides categorized access to all available components, organized into categories such as Chat Models, LLMs, Prompts, Memory, Chains, Agents, Tools, Document Loaders, Vector Stores, Embeddings, and Utilities. This categorization helps users quickly find relevant components without needing to know their exact names.

6.3 Demonstration: Simple RAG Chatbot Construction

To verify full platform functionality, a simple Retrieval-Augmented Generation (RAG) chatbot was constructed using the following components:

11. PDF File Loader — to ingest a PDF document as a knowledge source
-

-
12. Recursive Character Text Splitter — to divide the document into manageable chunks
 13. OpenAI Embeddings — to convert text chunks into vector representations
 14. In-Memory Vector Store — to store and retrieve document vectors
 15. ChatOpenAI — as the conversational AI model
 16. Conversational Retrieval QA Chain — to connect retrieval with generation

The construction of this complete RAG chatbot required no code to be written. The entire workflow was assembled through drag-and-drop interactions in approximately 10 minutes, demonstrating the core value proposition of the platform.

6.4 Performance Observations on 4GB RAM System

Given the hardware constraints of the development system (4GB RAM), several performance observations were noted:

- Initial startup time: Approximately 45-60 seconds due to loading all component libraries and initializing the database.
- UI responsiveness: The interface remained responsive during normal use, though some latency was observed when loading large component lists.
- Build time: The full pnpm build process took approximately 8-12 minutes, reflecting the computational demands of compiling a large TypeScript codebase.
- Runtime memory usage: The running Flowise server consumed approximately 600-800 MB of RAM, leaving sufficient memory for browser operation on a 4GB system.

These observations indicate that Flowise is operable on modest hardware configurations, though optimal performance would be achieved with 8GB or more of RAM for development purposes.

6.5 Evaluation Against Study Objectives

Evaluating the results against the objectives established in Chapter 1:

Objective	Status	Evidence
Academic review of LCNC technology	✓ Achieved	Chapters 2 and 3 provide comprehensive coverage
Flowise case study analysis	✓ Achieved	Chapter 4 covers architecture, features, and technology stack
Installation documentation	✓ Achieved	Chapter 5 documents complete step-by-step process
Capability evaluation	✓ Achieved	Chapter 6 demonstrates RAG chatbot construction
Practical understanding demonstration	✓ Achieved	Hands-on installation, version resolution, and testing

Chapter 7: Conclusion and Future Work

7.1 Conclusion

This report has presented a comprehensive academic exploration of AI-Powered No-Code and Low-Code platforms as a defining technology trend of 2026. Through theoretical analysis, literature review, and hands-on practical experimentation with the Flowise open-source platform, several important conclusions have been established.

First, the codeless revolution represents a genuine and profound transformation in how software is created. The combination of visual development environments and artificial intelligence has removed the primary barrier to software creation — the requirement for programming expertise — and replaced it with accessible, intuitive interfaces that any motivated individual can use productively.

Second, Flowise demonstrates that open-source platforms in the no-code AI space have reached a level of maturity and capability that rivals commercial alternatives. The platform's extensive component library, active community, and regular updates position it as a viable choice for serious AI application development in academic, research, and commercial contexts.

Third, the practical installation and testing process conducted for this report reveals that while the setup of such platforms requires some technical knowledge — particularly regarding version management and system configuration — the day-to-day use of the platform requires no coding expertise. This confirms the platform's value proposition as an accessible tool for citizen developers.

Fourth, the broader implications of this technology for developing nations such as Ethiopia are significant. Platforms like Flowise enable students, entrepreneurs, and organizations to build sophisticated AI-powered applications without the cost of hiring specialized developers, potentially accelerating digital transformation in contexts where technical talent is scarce.

7.2 Limitations

This study acknowledges several limitations. First, the practical testing was conducted on a single hardware configuration with limited RAM, which may not represent optimal performance conditions. Second, the study did not include a comparative evaluation against other no-code platforms, which would provide a more complete picture of the ecosystem. Third, the security and scalability aspects of self-hosted Flowise deployments were not evaluated in depth.

7.3 Future Work

Several directions for future work emerge from this study:

- **Comparative Analysis:** A systematic comparison of Flowise against other open-source no-code platforms such as n8n, Langflow, and Dify would provide valuable insights into the relative strengths and weaknesses of different architectural approaches.
 - **Enterprise Deployment Study:** Investigating the challenges and best practices for deploying Flowise at enterprise scale, including database configuration, load balancing, security hardening, and monitoring.
 - **Educational Applications:** Exploring the use of Flowise as a teaching tool in university courses on artificial intelligence, enabling students to prototype AI systems without barriers to entry.
-

-
- **Local Language Integration:** Investigating the integration of Amharic and other Ethiopian languages into Flowise-based AI systems, enabling AI applications that serve local communities in their native languages.
 - **Custom Component Development:** Extending the Flowise platform with custom-built components designed for specific Ethiopian industry needs, such as agricultural data analysis or healthcare information systems.

7.4 Final Reflection

The Codeless Revolution is not merely a technological trend — it is a democratization movement that has the potential to fundamentally reshape who participates in the creation of technology. As AI becomes embedded in the fabric of no-code platforms, the gap between having an idea and having a working application continues to shrink. For students at institutions like AASTU, mastering these platforms represents not just academic preparation but genuine professional capability in the technology landscape of 2026 and beyond.

References

The following references were consulted in the preparation of this report:

- FlowiseAI. (2026). Flowise: Build LLM Apps Easily. Official Documentation. <https://docs.flowiseai.com>
- FlowiseAI. (2026). Flowise GitHub Repository. <https://github.com/FlowiseAI/Flowise>
- Gartner. (2026). Magic Quadrant for Enterprise Low-Code Application Platforms. Gartner Research.
- Forrester Research. (2026). The Forrester Wave: Low-Code Development Platforms For Professional Developers. Forrester.
- Richardson, C., & Rymer, J. R. (2021). Now Tech: Low-Code Development Platforms, Q1 2021. Forrester Research.
- Rokis, K., & Kirikova, M. (2022). Challenges of Low-Code/No-Code Software Development: A Literature Review. Databases and Information Systems, 2022.
- Sahay, A., Indamutsa, A., Lo, D., & Nguyen-Duc, A. (2020). Supporting the understanding and comparison of low-code development platforms. Proceedings of the 46th Euromicro Conference on Software Engineering and Advanced Applications.
- Microsoft. (2026). Microsoft Power Platform Documentation. <https://learn.microsoft.com/power-platform>
- Bubble. (2026). Bubble — Build Web Apps Without Code. <https://bubble.io>
- OutSystems. (2026). OutSystems Platform Documentation. <https://www.outsystems.com/platform>
- Zapier. (2026). Zapier Automation Platform. <https://zapier.com>
- Addis Ababa Science and Technology University. (2026). Official University Website. <https://www.aastu.edu.et>
- LangChain. (2026). LangChain Documentation. <https://python.langchain.com>
- World Economic Forum. (2023). Future of Jobs Report 2023. World Economic Forum.
- IDC. (2026). Worldwide Low-Code Development Technologies Forecast. International Data Corporation.
- FlowiseAI. (2026). Flowise Cloud Platform — Live Demo and Verification. <https://cloud.flowiseai.com> (Instructors may visit this link to view and verify the live running version of the Flowise platform that was studied and tested in this report.),
-